

CI/CD Pipeline for Kubernetes Deployment

Key Alignment Notes

This document reflects the final report details: Jenkins runs as a Kubernetes workload Pod and is exposed via NodePort 32000; the application is exposed via NodePort 30080; the Jenkins Deployment uses the custom image `grafanaw/jenkins\_custom:its` that includes Docker CLI and kubectl, and mounts the host Docker socket for Docker operations.

Architecture Overview

```
Developer → GitHub → Jenkins (Pod in Kubernetes)
→ Docker Image Build & Push (via Docker socket mount)
→ Docker Hub
→ Kubernetes Deployment (2 Replicas)
→ Application Service (NodePort 30080)
→ Prometheus → Grafana
```

1. Kubernetes Environment Verification (Docker Desktop)

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
kubectl config current-context
kubectl config get-contexts
```

2. Deploying the Application on Kubernetes (NodePort 30080)

```
kubectl apply -f k8s/

kubectl get deployments -n asdn
kubectl get pods -n asdn
kubectl get svc -n asdn

# Application Service: asdn-project-service
# Type: NodePort
# NodePort: 30080
# Access: http://localhost:30080
```

3. Testing the Application Inside the Cluster

```
kubectl exec -it -n asdn <pod-name> -- sh

kubectl run tmp-curl --rm -it --restart=Never --image=curlimages/curl -- sh

curl http://asdn-project-service/health
curl http://asdn-project-service/echo
curl http://asdn-project-service/info
```

4. Docker – Containerization

```
docker build -t grafanaw/asdn_project:latest .
docker images
docker login
docker push grafanaw/asdn_project:latest
```

5. Git + GitHub

```
git add .
git commit -m "Update application"
git push
```

6. Jenkins (Running inside Kubernetes Cluster) — NodePort 32000

```
kubectl get pods -n jenkins
kubectl get svc -n jenkins
kubectl get deployments -n jenkins

# Jenkins Service: jenkins-service (NodePort)
# NodePort: 32000
# Access: http://localhost:32000

kubectl logs -n jenkins deploy/jenkins

kubectl rollout restart deployment jenkins -n jenkins
kubectl rollout status deployment jenkins -n jenkins

# (Optional) Port-forward alternative
kubectl -n jenkins port-forward svc/jenkins-service 8080:8080

kubectl exec -it -n jenkins <jenkins-pod-name> -- sh
```

7. Jenkins Custom Image Used (Controller + Pipeline Runtime)

```
# Jenkins Deployment uses a custom image that includes required tools:
# Image: grafanaw/jenkins_custom:lts
# Includes: Docker CLI + kubectl
# Mounts: /var/run/docker.sock (host Docker socket) to build/push images from Jenkins pod
```

8. Jenkins Pipeline Commands (Executed Inside Jenkins Pod)

```
# Build Stage
docker build -t grafanaw/asdn_project:<BUILD_NUMBER> -t grafanaw/asdn_project:latest .

# Test Stage
docker run --rm -w /app -e PYTHONPATH=/app grafanaw/asdn_project:<BUILD_NUMBER> pytest -q

# Push Stage
docker login -u <username> --password-stdin
docker push grafanaw/asdn_project:<BUILD_NUMBER>
docker push grafanaw/asdn_project:latest

# Deploy Stage (using kubeconfig secret file credential in Jenkins)
kubectl apply -f k8s/ --validate=false
kubectl rollout status deployment asdn-project -n asdn
```

9. Monitoring – Prometheus & Grafana

```
kubectl create namespace monitoring

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring

kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

10. Accessing Grafana

```
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80

Access:
http://localhost:3000
```

11. Final Validation

```
kubectl get pods -A
kubectl get deployments -A
kubectl get svc -A

kubectl rollout status deployment asdn-project -n asdn
```